

Redes de Ordenadores

GII – IYA022

Comunicación de datos

Mariano Benito



Universidad
Europea
del Atlántico

- Principios básicos
- Transmisión asíncrona y síncrona
- Detección y corrección de errores
 - Mecanismos de detección
 - Mecanismos de corrección
- Control de flujo
- Control de errores

- Comprender cómo se organiza la información en la **capa de enlace**.
- Describir y comprender el **control de errores**.
- Describir y comprender el **control de flujo**.

Aplicación

Transporte

Red

Enlace

Física

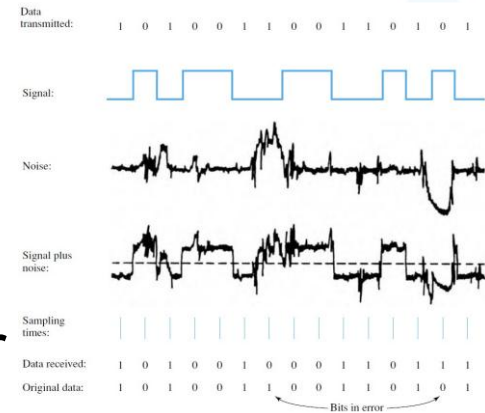
- A nivel de enlace, dos dispositivos están conectados por un canal que, a efectos prácticos, se comporta como un cable:



- El objetivo de la capa de enlace es proporcionar un **enlace físico fiable** entre dispositivos. Sus principales funciones son:
 - **Conectar y desconectar el enlace.**
 - Agrupar los bits en bloques llamados **tramas** (delimitación) y definir la estructura de estas.
 - **Asignar direcciones** de origen y destino a cada trama.
 - **Detectar** y, en algunos casos, **corregir errores**.
 - **Controlar el flujo** entre emisor y receptor para evitar su saturación.

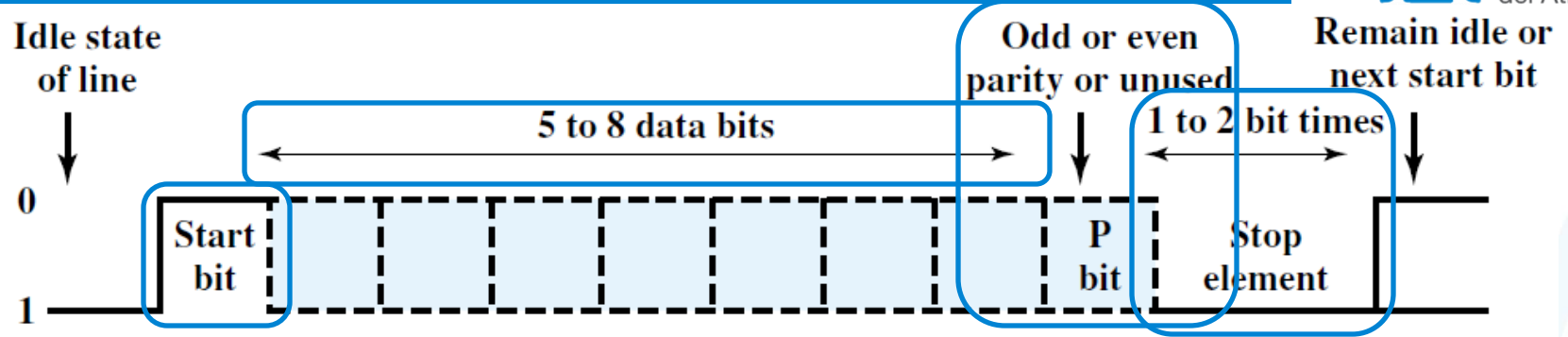
- En general, la capa de enlace ofrece tres tipos de servicio:
 1. No orientado a conexión y sin acuse de recibo.
 2. No orientado a conexión con acuse de recibo.
 3. Orientado a conexión con acuse de recibo.

- Para muestrear los símbolos recibidos correctamente se debe conocer el instante de llegada y su duración.
- Emisor emite a 1 Mbps (1 bit cada 1 μ s).
- Receptor intenta muestrear en la parte central de cada bit (cada 1 microsegundo).
- Reloj del emisor $\neq$ Reloj del receptor.
- Si el error de sincronismo es del 1%, el primer muestreo se desplaza 0,01 μ s.
- Tras 50, $50 \times 0,01 = 0,5 \mu$ s $\rightarrow$ error al muestrear el siguiente símbolo.

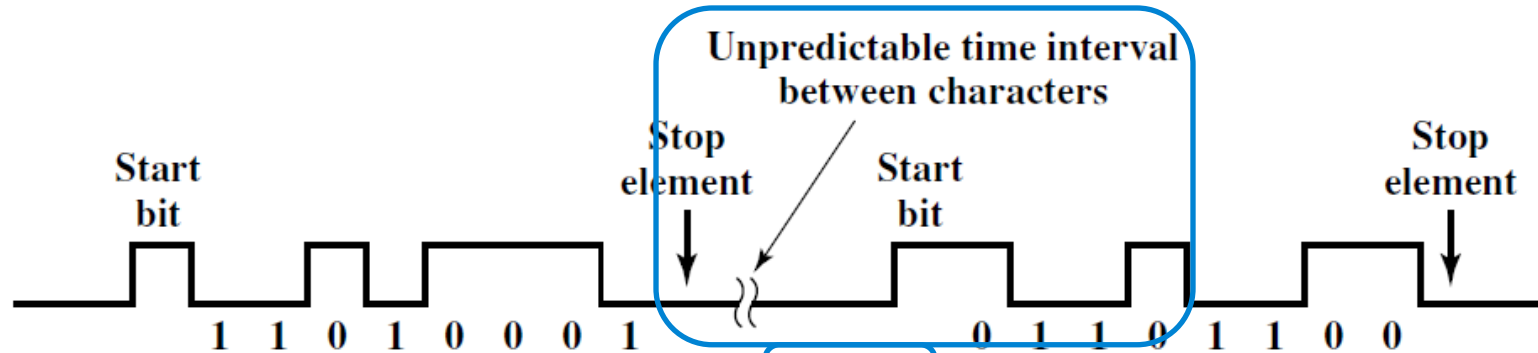


Dos técnicas principales para resolverlo: transmisión asíncrona y transmisión síncrona

- Transmisión en la que **los datos se envían carácter a carácter** (5 a 8 bits).
- El receptor se resincroniza al principio de cada carácter.
- **Delimitación del carácter:** intervalos de comienzo y final.
- **Paridad:** opcionalmente, se añade un cambo de paridad (par/impar) para detectar errores.
- **Características principales:**
 - Mecanismo sencillo y de bajo coste
 - *Overhead* grande: requiere entre 2 y 4 bits adicionales por carácter.
 - Adecuado solo para datos con vacíos grandes (teclado).

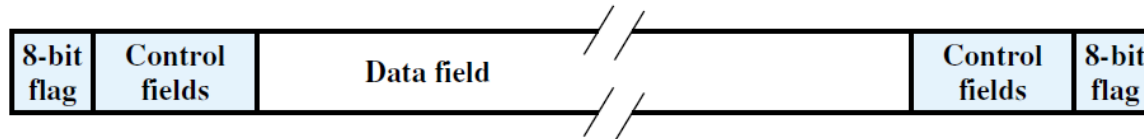


(a) Character format



(b) 8-bit asynchronous character stream

- En transmisión síncrona **se agrupan los datos en bloques denominados tramas (*frames*)**.
- **Delimitación:**
 - Cada trama comienza con un patrón de bits característico llamado **preámbulo (*preamble*)**.
 - **El final** puede determinarse de tres formas:
 1. Utilizando un **delimitador** especial similar al preámbulo.
 2. Estableciendo una **longitud fija** de trama.
 3. **Indicando su tamaño** en un campo de control.
- Para bloques mínimamente grandes, la transmisión síncrona es mucho más eficiente que la asíncrona.



- Problema: **¿Qué ocurre si el delimitador que indica el final aparece en los datos del usuario?**
 - Podría confundirse con el final de la trama. Sin embargo, no puede evitarse que el usuario transmita ciertas secuencias (**transparencia en los datos**).
- Solución: uso de **inserción de bits** o bytes (*bit/byte stuffing*).
 - Cada vez que en el transmisor aparece esa secuencia, se introduce un bit para evitar que aparezca. El receptor detecta y elimina ese bit adicional.
 - HDLC emplea el byte 01111110 como *flag*.
 - El transmisor inserta un 0 cuando encuentra 5 1s seguidos.
 - Cuando el receptor detecta 5 1s:
 - Si el 6º bit es 0, se elimina.
 - Si el 6º bit es 1 y el 7º un 0, se acepta el *flag* como fin de trama.
 - Si el 6º bit es 1 y el 7º un 1, el emisor indica *abort*.

Original pattern:

111111111111011111101111110

After bit-stuffing:

1111101111101101111101011111010

- **Error:** alteración de un bit entre su transmisión y recepción.
- Suelen clasificarse en:
 - **Errores aislados:** afectan únicamente a un bit, sin que los bits cercanos resulten alterados.
 - La probabilidad de que un bit recibido sea erróneo se denomina tasa de error por bit (*Bit Error Rate*, BER) y se representa como P_b .
 - Probabilidad de que una trama de F bits llegue sin errores: $(1 - P_b)^F$.
 - **Errores a ráfagas:** una ráfaga de longitud B se define como un grupo de B bits contiguos en el que el primero, el último y un número cualquiera de bits intermedios han resultado erróneos.

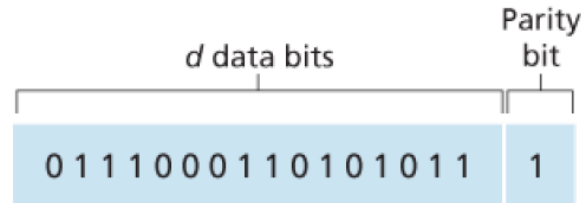
- El control de errores introduce redundancia en los datos, lo que obliga a encontrar un equilibrio entre la **eficiencia** y la **fiabilidad**.
- Existen dos tipos de códigos:
 - Códigos **detectores** de errores
 - Códigos **correctores** de errores.
- Los correctores requieren un mayor *overhead* aunque su uso resulta especialmente conveniente en canales ruidosos o propensos a errores, puesto que las **retransmisiones** tienen la misma probabilidad de resultar defectuosas.

- El NIF español es un ejemplo de código detector ya que la letra se obtiene a partir de la operación módulo 23 (resto al dividir entre 23) del número.
- A cada resultado, entre 0 y 22, se le asigna una letra (que es más difícil de confundir que un número) según esta tabla:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
T	R	W	A	G	M	Y	F	P	D	X	B	N	J	Z	S	Q	V	H	L	C	K	E

- Envío triplicado de los bits a transmitir:
 - 0 1 1 0 → 000 111 111 000
 - Puede funcionar en “modo de detección” o “modo de corrección”.
 - “**falso negativo**” → cuando dice **incorrectamente** que **no** hay error.
 - Modo de detección: permite detectar 1 o 2 errores:
 - 000 111 101 000 → Detecta el error
 - 000 111 100 000 → Detecta el error
 - 000 111 000 000 → ¡falso negativo!
 - Modo de corrección: permite corregir 1 error:
 - 000 111 101 000 → Corrige a 000 111 111 000
 - 000 111 100 000 → Corrige a 000 111 000 000 ¡mal!

- **Paridad:** bit añadido tal que el carácter resultante (datos + paridad) tenga un número impar (paridad impar) o par (paridad par) de unos.
 - **Falso negativo:** cuando dos (o un número par) de bits se invierten.

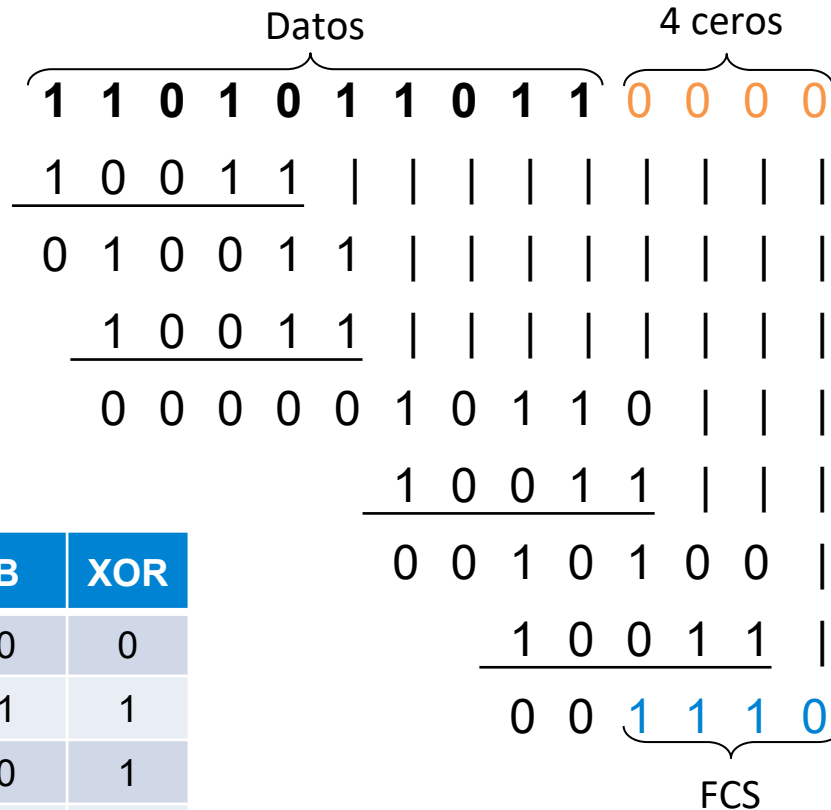


- **Suma de verificación (*checksum*):** resultado de la aplicación de una función sobre los datos del mensaje que se concatena al mensaje. Si en el receptor, la misma operación da otro resultado, se concluye que la transmisión tiene errores.

- **Idea:** verificar que la secuencia enviada sea divisible entre un valor dado, interpretando los **datos** como los coeficientes de un polinomio $M(x)$ de grado $k - 1$ para una trama de k bits.
- Polinomio generador $G(x)$: secuencia binaria conocida de antemano utilizada para calcular un resto. Ejemplos:
 - $G(x) = x^8 + x^7 + x^3 + x^2 + 1 \rightarrow 110001101$
 - $G(x) = x^{16} + x^{12} + x^5 + 1 \rightarrow 10001000000100001$
- **Secuencia de comprobación de la trama** (*Frame Check Sequence*, FCS): secuencia de longitud igual al grado del polinomio generador tal que la concatenación de los datos y la FCS es divisible por $G(x)$.
- La operación suma y resta en aritmética de polinomios en un cuerpo binario (módulo 2) es equivalente a una XOR sin acarreos.
- **Ventajas:**
 - Gracias a las propiedades de la aritmética polinómica se logra detectar eficazmente un gran número de patrones de error.
 - El cálculo es especialmente sencillo de implementar en hardware digital, lo que permite realizar la verificación sin afectar el rendimiento de las comunicaciones.

1. Siendo r el grado de $G(x)$, el emisor concatena r bits cero al mensaje $M(x)$, formando la cadena c_1 que contiene $k + r$ bits.
2. El emisor divide c_1 entre $G(x)$ y obtiene el resto (FCS).
3. El emisor resta la FCS a la cadena $c_1 \rightarrow$ se sustituyen los últimos bits de c_1 por la FCS y se envía.
4. El receptor divide la cadena recibida entre $G(x)$.
 - $Resto = 0 \rightarrow$ no ha habido errores.
 - $Resto \neq 0 \rightarrow$ detecta errores en la transmisión.
 - **Falso negativo:** solo ocurre si la secuencia de errores es un múltiplo del polinomio generador.
 - Se detecta cualquier ráfaga de errores más corta que el grado del polinomio generador.

Ejemplo de CRC



$$G(x) = x^4 + x + 1$$

$$\begin{array}{r} 1 \ 0 \ 0 \ 1 \ 1 \\ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \end{array}$$

El **emisor** concatena el FCS obtenido (1110) al mensaje y genera la cadena a transmitir: **11010110111110**.

El **receptor** divide la cadena recibida entre $G(x)$ y el resto es 0 si no ha habido errores.

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

- Existen 2 mecanismos generales para corregir errores:
 - **Detectar errores y retransmitir** (Backward Error Correction, BEC)
 - **Envío de información redundante** para que el receptor pueda detectar y corregir sin retransmitir (Forward Error Correction, FEC).
- FEC es más apropiado en:
 - Aplicaciones inalámbricas: la tasa de errores por bit puede ser elevada, lo que generaría un gran número de retransmisiones.
 - Enlaces por satélite: el retardo de propagación es muy elevado comparado con el tiempo de transmisión de la trama.
 - Enlaces simplex: TDT, radio digital, ...

- Mecanismo que **regula la transmisión** en un enlace **para evitar que el transmisor sobrecargue al receptor**, generando desbordamientos de su memoria temporal (*buffer*).
 - El transmisor es más rápido que la velocidad de procesamiento del receptor.
 - El receptor es un equipo de red y el siguiente enlace es más lento o está saturado.
- En general, podemos considerar 4 alternativas:
 1. **Ausencia de control de flujo** → ¡enlace no fiable!
 2. Control de flujo mediante **pausas**
 - Control de flujo mediante confirmaciones:
 3. **Parada y Espera** (*Stop & Wait*)
 4. **Ventana deslizante**

- T_{tx} : **tiempo de transmisión**, tiempo empleado para emitir todos los bits de una trama sobre el medio (s).
- V_{tx} : **velocidad de transmisión**, velocidad a la que el transmisor emite bits al medio (bits/s).
- d , **longitud del enlace** (m).
- L , **longitud** (promedio) **de una trama** (bits).
- T_p o R_p : **tiempo** (o retardo) **de propagación**, tiempo empleado por un bit en atravesar el medio (s), $T_p = \frac{d}{V_p}$.
- B : capacidad del enlace medida en bits, representa cuántos bits se envían hasta que el primero llega al receptor, $B = V_{tx} \cdot T_p$.
- a : capacidad del enlace medido en tramas, $a = \frac{B}{L}$.
 - Si $a < 1$, las tramas llegan al receptor antes de haberse completado su envío.
 - Si $a > 1$, pueden coexistir varias tramas en el medio (“en vuelo”).

- **Si no hay control de flujo, el enlace no es fiable**, ya que se pueden “perder” tramas sin necesidad de errores de transmisión.
- Para conseguir un enlace fiable, otra capa tendrá que hacer control de flujo y de errores.
 - En TCP/IP: Ethernet (nivel 2) por defecto no tiene control de flujo. A nivel de transporte (nivel 4):
 - TCP implementa control de flujo y de errores, mediante ventana deslizante y *checksums*.
 - UDP no implementa control de flujo. Se usa en enlaces en los que no se pueden repetir tramas, por ejemplo, para *streaming*.

- También llamado **ON/OFF signaling** o **Xon/Xoff**
- **El receptor envía un mensaje de “PAUSA” cuando ve que su búfer (almacenamiento temporal) se va a llenar.**
 - Ejemplo: Ethernet con el anexo 802,3x, donde se utiliza un mensaje de PAUSE que indica la duración de la pausa.
- **La pausa debe enviarse antes de que se llene el búfer, pero ¿cuánto antes?:**
 - Cuando se envía la pausa hay B bits en vuelo.
 - Cuando llega la pausa al transmisor, este ha enviado otros B bits:
 - El mensaje de pausa tarda T_p segundos.
 - Durante ese T_p el transmisor envía (como máximo) $T_p \cdot V_{tx} = B$ bits.
 - Umbral mínimo: $2B = 2 \cdot T_p \cdot V_{tx}$ bits disponibles en el búfer.
 - Por esto, el *buffer* debe tener al menos $2B$ bits si se quiere emplear Pausas.

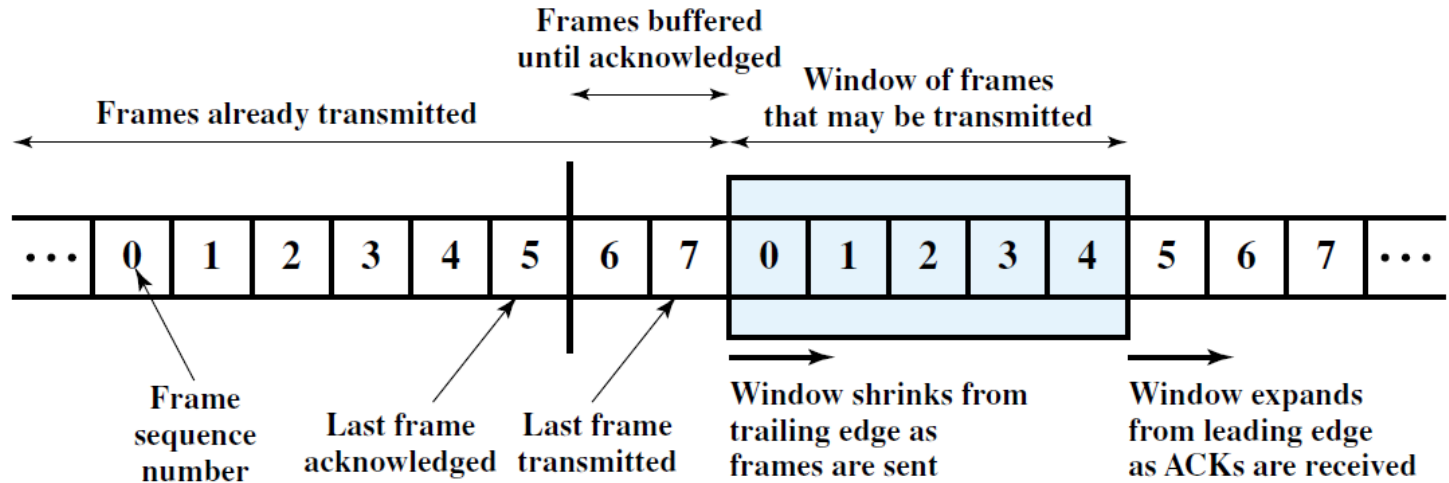
- **El receptor envía una confirmación (*ACKnowledgement*) por cada trama que recibe:**
 - El transmisor espera el **ACK** antes de enviar otra trama nueva.
 - El receptor puede parar el flujo de datos no enviando confirmaciones.
- **Diferencias con el control de flujo basado en Pausas:**
 - El transmisor sabe cuál es el búfer que tiene disponible el receptor en cada momento: el espacio para una o ninguna trama.
 - Además, **el ACK implica una confirmación de recepción correcta**, pero con Pausas no se sabe si se corrompieron las tramas.
- Es efectivo con tramas de gran tamaño respecto a la longitud del enlace, como en enlaces de baja velocidad de transmisión o corta distancia.
- Con tramas pequeñas (respecto a la capacidad del enlace) el medio de transmisión casi siempre estaría desocupado.

Principal limitación: solo puede haber una trama en tránsito en el enlace, lo que restringe seriamente la utilización del canal en escenarios con una velocidad de transmisión alta o largas distancias.

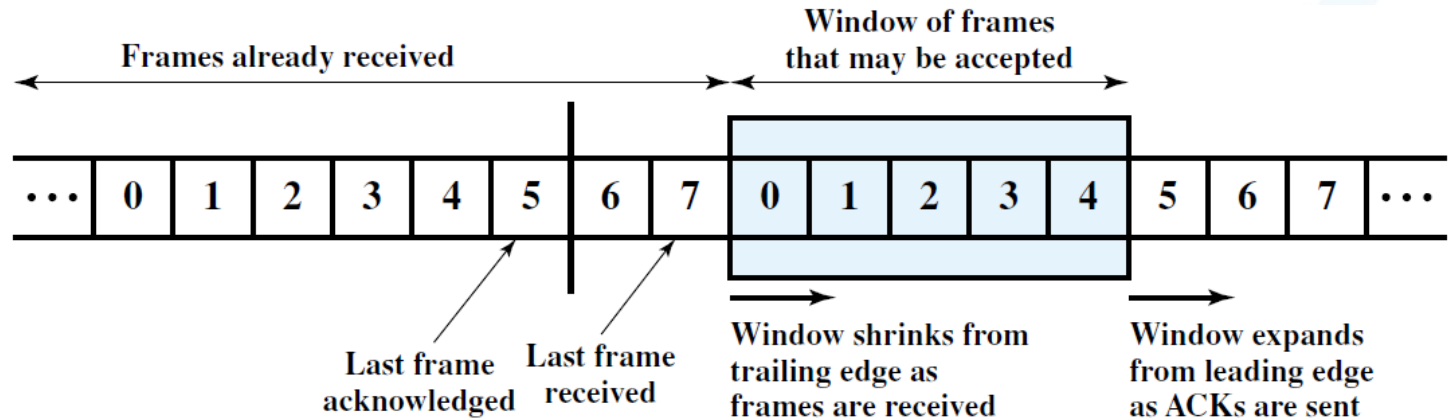
- **El transmisor puede enviar hasta W tramas antes de recibir una confirmación.** El receptor debe tener un búfer suficiente para almacenar W tramas.
- **Cada trama se etiqueta con un número de secuencia:**
 - Con k bits, las tramas se numeran módulo 2^k : $0, 1, 2, \dots, 2^k - 1$.
- **Ventana deslizante con $W = 1$ es igual a Parada y espera.**
- **El mensaje de confirmación puede definirse de 2 maneras:**
 - **ACK:** se incluye el número de secuencia de la última trama recibida correctamente.
 - **RR (*Receiver Ready*):** se incluye el número de secuencia de la siguiente trama que se espera recibir.
- **El tamaño máximo de la ventana está limitado por:**
 - El tamaño del búfer del receptor.
 - El tamaño del campo de numeración en la cabecera de la trama. Para implementar una ventana W debe ser $W \leq 2^k - 1$:
 - No puede ser 2^k por si ocurren errores, (*lo vemos luego*).
 - Para un tamaño de ventana W , el campo debe tener al menos $\log_2[W + 1]$ bits.

$$W = 7$$

Transmisor

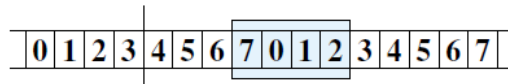
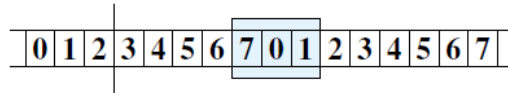
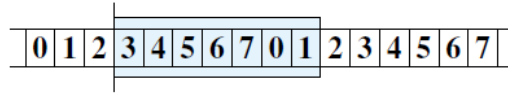
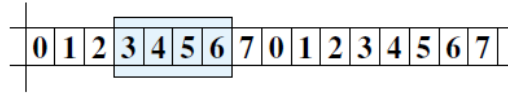
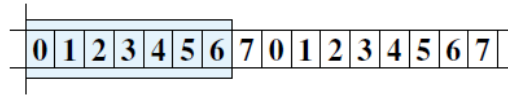


Receptor



$W = 7$

Transmisor



$F0$

$F1$

$F2$

$RR\ 3$

$F3$

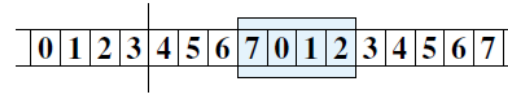
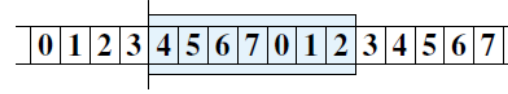
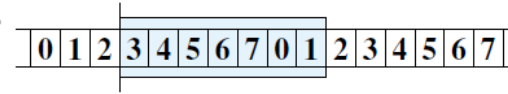
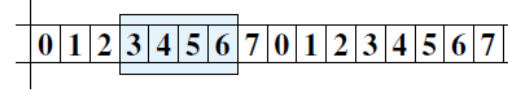
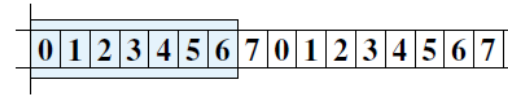
$F4$

$F5$

$F6$

$RR\ 4$

Receptor

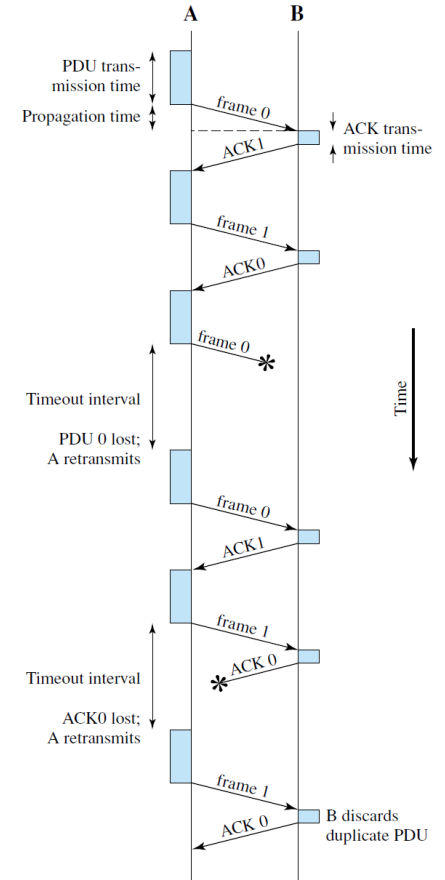


- **Receptor no preparado** (*Receive Not Ready, RNR*): el receptor puede confirmar las tramas, pero se prohíbe la transmisión de tramas adicionales.
 - Evita retransmisiones innecesarias.
 - Necesario cuando el receptor no es capaz de procesar los datos a la velocidad de recepción. Este deberá transmitir una confirmación normal que “reabra” la ventana cuando pueda procesar más tramas.
- **Incorporación de confirmación** (*piggybacking*): si la transmisión de tramas se realiza en dos direcciones:
 - Las tramas de datos de un nodo llevan la confirmación de las tramas del otro en la cabecera.
 - Si no hay datos que enviar, se utiliza una trama de confirmación explícita desencadenada por un temporizador para que el transmisor no piense que se han perdido datos al no recibir la confirmación.
- **Confirmaciones acumulativas**: cuando se envía una confirmación para un índice, se está confirmando esa trama y las anteriores.
 - Útil en caso de pérdida de alguna trama ACK.
- Alternativamente, **se puede hacer el control de flujo a nivel de bytes**.
 - Tiene sentido especialmente si las tramas son de tamaño variable.
 - Utilizado en TCP (nivel de transporte, no de enlace, pero misma idea).

- **Conjunto de mecanismos empleados para detectar y solventar errores de transmisión de tramas en un enlace.**
 - Idea: el transmisor añade información adicional (overhead) para el tratamiento de errores:
 - Paridad, *checksum*, CRC → detección en destino.
 - FEC → auto-corrección en destino.
- Si se pierde una trama completa, se detecta mediante el número de secuencia.
- Diferentes opciones para el tratamiento de errores:
 - a) Corrección mediante información redundante (FEC) (opcional)
Si no hay FEC, o aún así, se reciben errores que no se corrigen:
 - b) Descarte de tramas incorrectas → implica pérdidas
 - c) Solicitar una retransmisión (*Automatic Repeat reQuest*, ARQ):
 - ARQ con Parada y espera.
 - ARQ con vuelta atrás N.
 - ARQ con rechazo selectivo (retransmisión selectiva).

- Es el **mecanismo de Parada y espera con retransmisiones**.
- Cada trama se confirma con un ACK.
- **Si la trama de datos recibida está corrupta, se descarta:**
 - El transmisor utiliza un temporizador.
 - Si no se recibe ninguna confirmación antes de que el temporizador expire, se reenvía la misma trama.
- **Problema:** si el ACK se corrompe, el transmisor no la reconocerá:
 - El transmisor reenviará la trama de datos.
 - El receptor recibe la misma trama repetida. Si la aceptase como válida, estaría aceptando datos duplicados.
 - Para evitarlo, **hay que utilizar dos números de secuencia**.

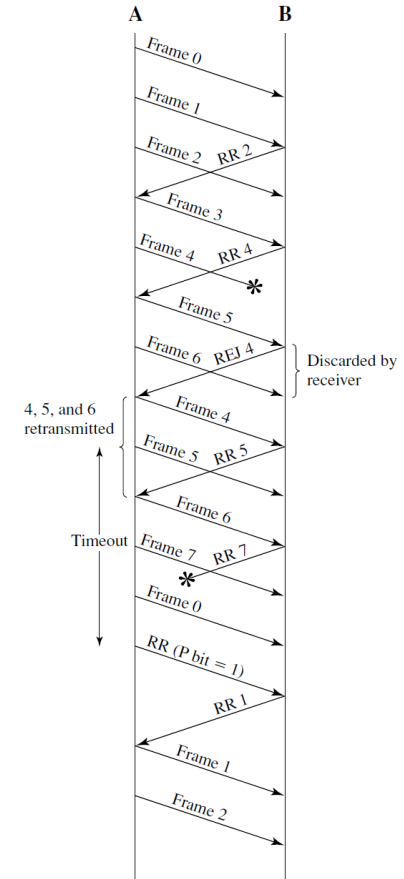
- Fuente:** Stallings (cap. 7)



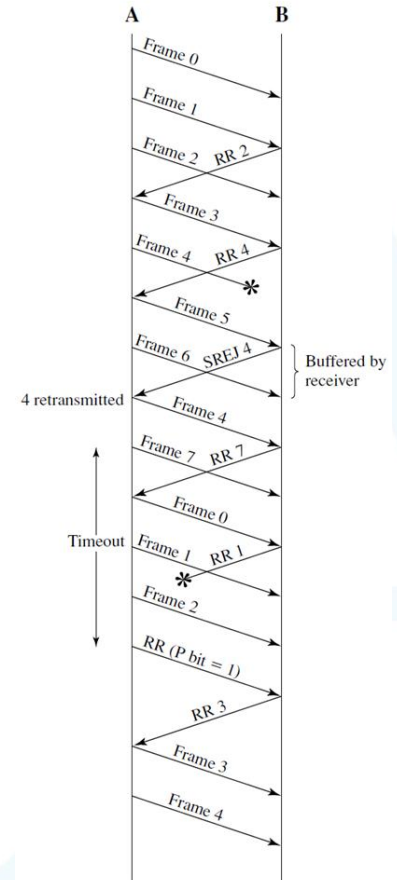
- **Es el mecanismo de ventana deslizante con retransmisión de tramas.**
 - Cuando hay un error, vuelve atrás la transmisión hasta el fallo, y retransmite.
- Se utiliza una ventana para controlar las tramas pendientes de confirmar.
 - **Requiere al menos $W + 1$ índices de numeración** para una ventana de W tramas para evitar ambigüedad en caso de errores. Con W índices, si se envían W tramas, una confirmación RR(0) ¿qué significa?
 - a) Se han recibido las W tramas y estoy esperando la siguiente, que vuelve a usar el índice 0, o
 - b) Se han perdido todas las tramas, y espero esperando la 1ª.
- **Si no hay errores, el receptor envía ACKs/RRs como antes.**
- **Si el receptor detecta errores:**
 - Las tramas corruptas o fuera de secuencia se descartan.
 - Se confirma siempre la última trama que si llegó correctamente.
 - Cuando llega la confirmación, el transmisor detecta que debe volver atrás y retransmitir desde la errónea en adelante.
- También **requiere un temporizador de retransmisión** para el caso de pérdida de tramas o confirmaciones.

- **Trama recibida con errores:**
 - El receptor detecta un error en la trama i y la descarta silenciosamente
 - Al estar corrupta, no podemos fiarnos de los campos de la cabecera
 - Después llega la $i + 1$ sin haber recibido la i .
 - Se envía una confirmación, indicando que se espera la i (RR(i)).
 - El transmisor debe enviar la trama i y todas las posteriores.
- **Trama perdida:** igual que antes, si se pierde la trama i .
- **Trama perdida, sin envío de tramas posteriores:**
 - La trama i se pierde o esta corrupta, pero no se envía la $i + 1$.
 - El receptor no la confirma.
 - El transmisor no recibe confirmación y al expirar el temporizador de retransmisión, reenvía la trama i .
- **Pérdida de la confirmación (ACK o RR):**
 - El transmisor no recibe confirmación (igual que antes) y reenvía los datos al expirar el temporizador.
 - El receptor detecta que las tramas son repetidas (usando el número de secuencia), envía una confirmación y las descarta.

- Al estar basado en ventana, **es más eficiente que parada y espera**, pero más complejo.
- Algunas opciones de implementación:
 - **Rechazo explícito (REJ)**: cuando llega una trama i corrupta, se envía $REJ(i)$, solicitando su retransmisión. Se acelera la retransmisión al no tener que esperar a la expiración de un temporizador.
 - **Solicitud de confirmación**: si al transmisor no le llega confirmación de alguna trama, en lugar de reenviarla, envía una solicitud de “número de orden” (Trama RR con un bit P, *poll*, a 1). El receptor responde con la trama que espera.
 - Si la trama de datos se había perdido, se comporta como antes.
 - **Si se había perdido la confirmación, se evita la retransmisión.**



- **ARQ con rechazo o repetición selectiva**, también basado en ventana deslizante.
- Análogo a vuelta atrás N, pero **el receptor envía una confirmación negativa** (*Selective Reject*, SREJ) cuando hay algún error. Solo se retransmiten las tramas corruptas.
- **El receptor acepta las tramas posteriores a la corrupta**, y las almacena en un búfer.
- **Con una ventana de W tramas, requiere $2W$ índices de secuencia** para evitar ambigüedades, frente a $W + 1$ con vuelta atrás N.
- Ventaja: **es más eficiente** dado que minimiza el número de retransmisiones.
- **Problemas:**
 - El receptor debe tener un búfer para la reordenación grande.
 - El transmisor requiere una lógica más compleja.
 - Precisa un campo de numeración mayor.



Redes de Ordenadores

GII – IYA022

Mariano Benito